

Módulo 2 — Árboles, bosques y boosting

Curso de Machine Learning para
estimación de pobreza e inequidad

Bloque teórico — Día 2

Día 2

Banco Mundial - INEC - USFQ

Agenda del bloque

1. Árboles CART y poda
2. Random Forest, OOB e importancia
3. Potenciación por gradiente (*gradient boosting*)
4. XGBoost y LightGBM
5. Búsqueda de hiperparámetros y evaluación
6. Mapa comparativo
7. Q&A

Al final del módulo el participante:

1. Explica criterios de impureza y poda
2. Entiende por qué el OOB es validación cruzada gratis
3. Interpreta importancias con sus limitaciones
4. Explica boosting como corrección secuencial de errores
5. Elige entre XGBoost y LightGBM con criterio
6. Distingue búsqueda en grilla, aleatoria y bayesiana de hiperparámetros

Árboles CART y poda

La idea: una secuencia de preguntas binarias

- Árbol = secuencia anidada de divisiones binarias
- Predicción constante en cada hoja
- No paramétrico, adaptativo
- Frontera siempre alineada a ejes (escalonada)

Criterio de división: ganancia de pureza

Para cada candidato (j, s) , dividir si:

$$\Delta i(t) = i(t) - \frac{n_L}{n_t} i(t_L) - \frac{n_R}{n_t} i(t_R) > 0$$

- Búsqueda **greedy**: localmente óptima, globalmente no
- Sin backtracking: nunca se vuelve atrás

Regresión: varianza intra-nodo

$$i(t) = \frac{1}{n_t} \sum_{i \in t} (y_i - \bar{y}_t)^2$$

Clasificación, K clases con proporciones $\hat{p}_k$:

$$i_G(t) = 1 - \sum_k \hat{p}_k^2 \quad (\text{Gini})$$

$$i_H(t) = - \sum_k \hat{p}_k \log \hat{p}_k \quad (\text{Entropía})$$

Gini $\approx$ entropía en práctica; ambas diferenciables y cóncavas.

- Hoja R_m : media (regresión) o moda / probabilidad (clasificación)
- **Invariante** a transformaciones monótonas
- No requiere escalamiento ni normalización
- Maneja categóricas y continuas en el mismo árbol
- Captura **interacciones** automáticamente al anidar splits

- Árbol completo $\rightarrow$ una hoja por observación $\rightarrow$ memoriza
- Varianza alta: pequeños cambios en datos $\rightarrow$ árbol distinto
- Solución: pre-poda (`min_n`, `max_depth`) o post-poda

Curva clásica: error de train cae monótono, error de validación tiene forma de U.

$$C_{\alpha}(T) = \sum_{m=1}^{|T|} \sum_{i \in R_m} L(y_i, \hat{y}_{R_m}) + \alpha |T|$$

- $\alpha \rightarrow 0$: árbol grande (sobreajusta)
- $\alpha \rightarrow \infty$: árbol pequeño (subajusta)
- Secuencia finita de subárboles encajados (Breiman et al. 1984)
- α se elige por validación cruzada (regla 1-SE)

- Alta varianza
- Fronteras escalonadas; no captura aditividad suave
- Sesgo en variables categóricas con muchos niveles

Esto motiva los ensambles: bagging, Random Forest, boosting.

Random Forest

Bagging = bootstrap aggregation (Breiman 1996):

- B datasets bootstrap $\rightarrow B$ árboles $\rightarrow$ promediar / votar
- Reduce varianza, mantiene sesgo

Si fueran independientes: $\text{Var}(\bar{Z}) = \sigma^2/B \rightarrow 0$.

La fórmula clave del bagging

En simple: promediar muchos árboles baja la varianza, pero solo hasta donde lo permita cuánto se parecen entre sí. Varianza del promedio de B variables con correlación par-a-par ρ :

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^B Z_b\right) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

- Bagging reduce el segundo término (B grande)
- RF reduce ρ aleatorizando variables en cada split

El piso de la varianza es $\rho\sigma^2$.

Random Forest: definición operativa

1. B muestras bootstrap
2. En cada split, muestrear m de p variables
3. Construir árbol completo sin podar
4. Agregar: promedio (regresión) o voto (clasificación)

Valores típicos: $m = \sqrt{p}$ (clasif.), $m = p/3$ (regresión).

Hiperparámetros y su efecto

Hiperparámetro	Efecto de ↑	Rango típico
<code>mtry</code>	Más correlación entre árboles	$\sqrt{p}$ o $p/3$
<code>trees</code>	Estabilidad, retornos decrecientes	500–2000
<code>min_n</code>	Árboles más superficiales	5–20

OOB: la idea de validación cruzada gratis

- Cada árbol b entrena en $\mathcal{D}^{*b}$
- $\approx 37\%$ de las obs. no están en cada muestra
- Esos OOB forman un “comité de validación” por observación

La derivación del 36.8 %

Probabilidad de que i no sea seleccionada en una muestra bootstrap:

$$\mathbb{P}(i \notin \mathcal{D}^{*b}) = \left(1 - \frac{1}{n}\right)^n \xrightarrow{n \rightarrow \infty} e^{-1} \approx 0,368$$

- Límite notable: $\lim(1 + a/n)^n = e^a$ con $a = -1$
- $n = 100 \rightarrow 0,366$; $n = 1000 \rightarrow 0,3677$

Cada observación queda OOB en $\approx 37\%$ de los árboles.

Para cada i , sea $\mathcal{B}(i) = \{b : i \notin \mathcal{D}^{*b}\}$:

$$\hat{y}_i^{\text{OOB}} = \text{agregar}(\{\hat{f}^{*b}(x_i) : b \in \mathcal{B}(i)\})$$

$$\hat{R}_{\text{OOB}} = \frac{1}{n} \sum_{i=1}^n L(y_i, \hat{y}_i^{\text{OOB}})$$

Aproximadamente insesgado del error de generalización (Breiman 2001).

- En RF se puede prescindir de CV externa
- Tuning de `mtry`, `min_n` directamente con OOB
- Útil en datasets grandes

Salvedad: para comparar RF vs. otros modelos, usar CV externa común a todos.

Importancia: disminución media de impureza (MDI)

$$\text{MDI}(j) = \frac{1}{B} \sum_{b=1}^B \sum_{t: \text{split en } j} \frac{n_t}{n} \Delta i(t)$$

- Subproducto del entrenamiento (gratis)
- **Sesgada** hacia variables con más categorías y mayor varianza (Strobl et al. 2007)

Importancia: permutación

Para cada árbol y cada X_j :

1. OOB error original
2. Permutar aleatoriamente X_j en OOB
3. Recalcular OOB error
4. Caída de desempeño = importancia

Más honesta, más cara. No depende de la cardinalidad.

Limitaciones de las importancias

- **Multicolinealidad**: la importancia se reparte arbitrariamente
- Importancia $\neq$ causalidad
- Inestabilidad bajo perturbaciones de datos
- Sin signo (no dice dirección del efecto)

Complemento natural: **SHAP** (Módulo 4).

- Baseline robusto, poco tuning
- OOB elimina necesidad de CV externa
- Importancias útiles pero con cautelas
- Limitación: no aprovecha estructura secuencial de errores

¿Y si los árboles se corrigen secuencialmente en vez de promediarse?

Potenciación por gradiente

La idea: corregir errores secuencialmente

Modelo aditivo:

$$\hat{F}_M(x) = \sum_{m=0}^M \nu h_m(x)$$

- Cada h_m corrige errores del ensamble previo
- “Aprendices débiles” (árboles pequeños)
- $\nu \in (0, 1] =$ tasa de aprendizaje

En iteración m :

$$h_m = \arg \min_h \sum_{i=1}^n L(y_i, \hat{F}_{m-1}(x_i) + h(x_i))$$

Intratable para pérdidas generales $\Rightarrow$ **gradiente funcional** (Friedman 2001).

En simple: cada nuevo árbol se entrena para corregir el error que dejó el ensamble anterior; ese “error a corregir” es el gradiente.

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=\hat{F}_{m-1}}$$

- L_2 : $r_{im} = y_i - \hat{F}_{m-1}(x_i)$ (residuo clásico)
- Log-pérdida con $F = \log\text{-odds}$: $r_{im} = y_i - \hat{p}_i^{(m-1)}$

Boosting ajusta el gradiente, no los residuos. El caso L_2 es solo un caso particular.

Algoritmo completo (Friedman 2001)

1. Inicializar $\hat{F}_0$ (constante)
2. Para $m = 1, \dots, M$:
 - 2.1 Calcular pseudo-residuos r_{im}
 - 2.2 Ajustar h_m a $\{(x_i, r_{im})\}$
 - 2.3 Opcional: γ_m por búsqueda lineal
 - 2.4 $\hat{F}_m = \hat{F}_{m-1} + \nu \gamma_m h_m$

- **Shrinkage** $\nu \in [0,01, 0,1]$
- **Stochastic boosting**: submuestrear filas (Friedman 2002)
- **Profundidad limitada** ($\text{depth} \in [3, 8]$)
- **Parada temprana** sobre M con validación

ν pequeño + M grande $>$ ν grande + M pequeño.

Comparación conceptual: RF vs. boosting

	Random Forest	Gradient Boosting
Estructura	Paralela	Secuencial
Reduce	Varianza	Sesgo
Sensibilidad a hp	Baja	Alta
Costo	Medio (paralelizable)	Alto (secuencial)

Cuándo RF es preferible

- Datasets pequeños (RF más estable)
- Pipelines con poco tiempo de tuning
- Requisito de OOB sin CV externa
- Producción donde el mantenimiento prima sobre el último 1 % de AUC

- Gradient boosting es el *marco general*
- XGBoost y LightGBM son *implementaciones con mejoras*
- Lo común: gradiente funcional + árboles

XGBoost y LightGBM

XGBoost: objetivo regularizado

Para un árbol h con T hojas y pesos w_j :

$$\mathcal{L}^{(m)} = \sum_{i=1}^n L(y_i, \hat{F}_{m-1}(x_i) + h(x_i)) + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

- γ : costo por hoja (complejidad estructural)
- λ : ridge sobre pesos de hoja

Aproximación de segundo orden (paso de Newton)

Taylor de segundo orden en torno a $\hat{F}_{m-1}$:

$$L(y_i, \hat{F}_{m-1} + h(x_i)) \approx L_0 + g_i h(x_i) + \frac{1}{2} h_i h(x_i)^2$$

con $g_i = \partial_F L$, $h_i = \partial_F^2 L$ evaluados en $\hat{F}_{m-1}$. Usa la curvatura, no solo el gradiente.

Sea I_j el índice de obs. en la hoja j :

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}$$

$$\mathcal{L}^*(T) = -\frac{1}{2} \sum_{j=1}^T \frac{(\sum_{i \in I_j} g_i)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T$$

Ganancia de split: si $\leq 0 \Rightarrow$ no dividir (γ regulariza).

Otros ingredientes de XGBoost

- Sparsity-aware splitting (dirección por defecto para faltantes)
- Histogram-based splits
- Paralelización por features
- Manejo nativo de valores faltantes
- Case weights y monotonicidad

LightGBM: tres innovaciones (Ke et al. 2017)

1. **Histogramas** con b bins (≈ 255): $O(n) \rightarrow O(b)$
2. **Crecimiento por hoja** (leaf-wise), no por nivel
3. **GOSS** (gradientes grandes) + **EFB** (variables excluyentes)

Leaf-wise puede sobreajustar en datasets pequeños $\Rightarrow$ `max_depth`, `num_leaves`.

XGBoost vs. LightGBM: criterios de elección

XGBoost Madurez, ecosistema, defaults conservadores

LightGBM Velocidad en datasets grandes, alta cardinalidad

CatBoost Mejor manejo nativo de categóricas

Hiperparámetros críticos en boosting

Hiperparámetro	Rango típico	Efecto
learning_rate (ν)	0.01–0.1	↓: estable, más iteraciones
max_depth	3–8	↑: más complejo
n_estimators	100–2000	con parada temprana
min_child_weight	1–10	control de hojas pequeñas
subsample	0.5–1.0	SGB
λ, γ	0–10	regularización explícita

Búsqueda de hiperparámetros y evaluación

El problema: muchos hiperparámetros, poco tiempo

En simple: cada combinación de hiperparámetros es un modelo distinto; hay que probar varias y quedarse con la mejor *en validación*.

- Cada candidato se evalúa por validación cruzada (Módulo 1)
- Probar todo es caro: el espacio crece exponencialmente
- La pregunta clave: **¿cómo explorar el espacio con criterio?**

Búsqueda en grilla vs. aleatoria

Grilla (`tune_grid`) probar todas las combinaciones de una rejilla fija

Aleatoria muestrear combinaciones al azar del espacio

Hallazgo clave (Bergstra & Bengio 2012): con pocos hiperparámetros que de verdad importan, la búsqueda **aleatoria** encuentra buenas soluciones más rápido que la grilla.

La idea: usar los resultados ya vistos para decidir *con inteligencia* qué combinación probar después.

1. Ajustar un *modelo sustituto* (p.ej. proceso gaussiano) del desempeño
2. Una *función de adquisición* equilibra explorar vs. explotar
3. Probar el punto más prometedor, actualizar, repetir

Menos evaluaciones que grilla o aleatoria para igual desempeño.

Clasificación F1 (balance precisión/recall), AUC-ROC, **PR-AUC** (mejor bajo desbalance)

Regresión RMSE (penaliza errores grandes), MAE (robusta a outliers)

Comparar todos los modelos con la MISMA métrica y el MISMO esquema de CV.

Síntesis y Q&A

Mapa comparativo de modelos basados en árboles

Modelo	Sesgo	Varianza	Costo	Interpret.	Hp sensibles
Árbol único	Alto	Alta	Bajo	Alta	depth, cp
Random Forest	$\approx$ árbol	Baja	Medio	Media	mtry, trees
XGB / LightGBM	Bajo	Baja c/reg.	Alto	Media (SHAP)	ν , depth, M

Regla práctica: empezar por RF, luego XGBoost/LightGBM si se persigue ceiling.

- Los árboles capturan no-linealidades e interacciones
- RF reduce varianza: bagging + aleatorización de variables
- OOB $\approx$ CV gratis
- Boosting reduce sesgo secuencialmente vía gradiente funcional
- XGBoost/LightGBM: implementaciones con regularización explícita

Módulo 3: SVM, k-NN y Naive Bayes

¿Preguntas?

-  Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *JMLR* 13.
-  Breiman, L. (1996). Bagging predictors. *Machine Learning* 24(2).
-  Breiman, L. (2001). Random forests. *Machine Learning* 45(1).
-  Breiman, L., Friedman, J.H., Olshen, R.A., & Stone, C.J. (1984). *Classification and Regression Trees*. Wadsworth.
-  Caruana, R., & Niculescu-Mizil, A. (2006). An empirical comparison of supervised learning algorithms. *ICML*.

-  Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *KDD 2016*.
-  Friedman, J.H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics* 29(5).
-  Friedman, J.H. (2002). Stochastic gradient boosting. *CSDA* 38(4).
-  Grinsztajn, L. et al. (2022). Why do tree-based models still outperform deep learning on tabular data? *NeurIPS D&B*.
-  Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.

-  Ke, G. et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. *NeurIPS 2017*.
-  Louppe, G. (2014). Understanding Random Forests. *PhD Thesis*, U. Liège.
-  Prokhorenkova, L. et al. (2018). CatBoost. *NeurIPS 2018*.
-  Quinlan, J.R. (1993). *C4.5: Programs for Machine Learning*. Morgan Kaufmann.
-  Strobl, C. et al. (2007). Bias in random forest variable importance measures. *BMC Bioinformatics* 8.